

Python Inheritance - Master Notes

Prepared by Thanesh

1. What is Inheritance?

Inheritance is an OOP concept in which a child class acquires the properties and methods of a parent class. It promotes code reusability, reduces duplication, simplifies maintenance, and models an IS-A relationship.

2. Advantages

- Code Reusability
- Easy Maintenance
- Extensibility
- Method Overriding
- Cleaner Architecture

3. Syntax

```
class Parent:
    pass

class Child(Parent):
    pass
```

Single Inheritance

One Parent -> One Child

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

d=Dog()
d.eat()
d.bark()
```

Multiple Inheritance

Multiple Parents -> One Child

```
class Father:
    def money(self): print("Money")
class Mother:
    def care(self): print("Care")
class Child(Father,Mother):
    pass
c=Child()
c.money()
c.care()
```

Multilevel Inheritance

Grandparent -> Parent -> Child

```
class A:
    def a(self): print("A")
class B(A):
    def b(self): print("B")
class C(B):
    def c(self): print("C")
obj=C()
obj.a(); obj.b(); obj.c()
```

Hierarchical

One Parent -> Many Children

```
class Animal:
    def eat(self): print("Eat")
class Dog(Animal):
    pass
class Cat(Animal):
    pass
```

Hybrid

Combination

```
class A: pass
class B(A): pass
class C(A): pass
class D(B,C): pass
```

Method Overriding

```
class Animal:
    def sound(self):
        print("Animal")
class Dog(Animal):
    def sound(self):
        print("Dog Barks")
Dog().sound()
```

super() Keyword

```
class Animal:
    def __init__(self):
        print("Animal Constructor")
class Dog(Animal):
    def __init__(self):
        super().__init__()
        print("Dog Constructor")
Dog()
```

Real-Time Banking Application Using Inheritance

Parent Class: BankAccount

Child Class: SavingsAccount

SavingsAccount inherits createAccount(), deposit(), withdraw(), and checkBalance(), and adds interest().

```
class BankAccount:
    def createAccount(self):
        self.accNo=input("Account No: ")
        self.name=input("Name: ")
        self.balance=float(input("Balance: "))
    def deposit(self):
        amt=float(input("Deposit: "))
        self.balance+=amt
    def withdraw(self):
        amt=float(input("Withdraw: "))
        if amt<=self.balance:
            self.balance-=amt
            print("Withdrawal Successful")
        else:
            print("Insufficient Balance")
    def checkBalance(self):
        print("Account:", self.accNo)
        print("Name:", self.name)
        print("Balance:", self.balance)

class SavingsAccount(BankAccount):
    def interest(self):
        self.balance+=self.balance*0.05
        print("5% Interest Added")

acc=SavingsAccount()

while True:
    print("\n1.Create Account")
    print("2.Deposit")
    print("3.Withdraw")
    print("4.Check Balance")
    print("5.Add Interest")
    print("6.Exit")
    choice=int(input("Enter Choice: "))
    if choice==1:
        acc.createAccount()
    elif choice==2:
        acc.deposit()
    elif choice==3:
        acc.withdraw()
    elif choice==4:
        acc.checkBalance()
    elif choice==5:
        acc.interest()
    elif choice==6:
        break
    else:
        print("Invalid Choice")
```

Explanation

1. BankAccount is the parent class.
2. SavingsAccount inherits all banking operations.
3. The child class adds only the interest() method.
4. The menu-driven program demonstrates practical inheritance and code reuse.

Interview Questions

- What is inheritance?
- Explain all five types.
- What is method overriding?
- What is super()?
- What is MRO?
- Difference between inheritance and composition?

One-Minute Interview Answer

Inheritance allows a child class to inherit the data members and methods of a parent class. It improves reusability, maintainability, and scalability. Python supports Single, Multiple, Multilevel, Hierarchical, and Hybrid inheritance. Common concepts include method overriding and the super() function.